

MPDG-SLAM: Motion Probability-based 3DGS-SLAM in Dynamic Environment

Conghao Huang¹ and Li Zhang^{2*} and Tianchen Deng³ and Kangxu Wang⁴ and Mingrui Li⁵

Abstract—We present MPDG-SLAM, a novel 3D Gaussian point cloud rendering SLAM method based on Motion Probability (MP) for dynamic interference handling. Current 3DGS-SLAM approaches for dynamic environments often rely on optical flow estimation masks. However, these deep learning-based optical flow models are computationally intensive and limited by processing speed, posing challenges for deployment on mobile devices in real-world scenarios. Moreover, existing systems depend on precise mask segmentation and corresponding loss functions for artifact removal, yet the pixel accuracy of optical flow estimation is constrained by real-world lighting conditions. To address these issues, we introduce a mobile-deployable Yolo and a mathematically derived Motion Probability (MP) attribute to label Gaussian points, which are then inversely mapped to the front-end feature tracking system to correct for dynamic object influences. By incorporating an MP-based penalty term, dynamic Gaussians corresponding to moving entities are explicitly removed to minimize their effect. Additionally, we design an edge warp loss based on MP estimation, enabling accurate artifact removal even with coarse segmentation masks. The experiments show that our approach notably improves the reconstruction quality of dynamic scenes, surpassing baseline methods and reaching speeds over 30 FPS on high-end GPUs, which suggests its potential for real-time use on mobile platforms after further optimization.

I. INTRODUCTION

Accurate mapping and self-localization in unknown environments are made possible in autonomous systems through the core technique known as Simultaneous Localization and Mapping (SLAM). 3D Gaussian Splatting empirically shows remarkable online execution efficiency and high-fidelity texture representation in novel view synthesis, which has spurred the development of dense SLAM systems based on 3D Gaussian Splatting [1], [2]. However, typical 3D Gaussian Splatting-based SLAM systems often assume static scenes, which perform well in virtual environments but lead to significant pose drift and dynamic artifacts when confronted with dynamic interference in real-world scenarios [3], [4]. To address these issues, some existing 3D Gaussian Splatting systems have adopted adaptive strategies. The core approach primarily involves using deep learning-based optical flow estimation systems to obtain masks of dynamic

objects [5], followed by employing a divide-and-conquer strategy to adjust the rendering function and introducing various complex rendering losses to remove dynamic elements. These methods generally face two main challenges: 1) Due to the current reliance of optical flow estimation systems on additional computational resources [6], they cannot be directly deployed on mobile platforms, thereby limiting the system's real-time performance. 2) To avoid introducing extra information, existing end-to-end systems depend on optical flow estimation masks for dynamic object removal and tracking correction. However, the accuracy of optical flow masks is constrained by lighting conditions and the state of moving objects in real-world environments, resulting in limited pixel matching accuracy. This leads to a decline in the quality of rendered artifacts and tracking precision, causing these methods to significantly lag behind traditional dynamic-resistant SLAM methods [7] in terms of ATE (Absolute Trajectory Error) metrics.

Therefore, this paper proposes MPDG-SLAM. We introduce the Motion Probability (MP) [8] attribute to label Gaussian points, enabling the classification of dynamic and static points in the backend without the need to directly remove dynamic features at the frontend. We adopt a Gaussian pyramid-based densification strategy, allowing frontend tracking features to directly initialize backend rendering Gaussians. By utilizing MP probability, we achieve simultaneous correction of dynamic Gaussian rendering and dynamic feature tracking. We deploy a YOLOv7 system [9] on mobile devices to provide semantic bounding boxes, and calculate MP values through a combination of instance segmentation and geometric methods. By integrating the MP attribute, we classify Gaussian points as dynamic or static, thereby improving pose estimation accuracy and reducing artifacts in the scene map. Subsequently, we employ an epipolar geometry-based motion state verification method to densify the map and recover static Gaussian points and corresponding feature points that were erroneously marked as dynamic. This achieves simultaneous optimization of tracking and rendering information, avoiding redundant decoupling processes. In 3D Gaussian rendering, we use MP to adjust photometric and depth rendering losses and introduce an MP-based penalty term to limit the impact of dynamic objects on rendering. Additionally, to promote better geometric alignment and mitigate noise in environments with dynamic content, we incorporate a novel edge transformation loss into the framework. Compared to other mask-based methods, our edge loss only requires a rough estimation of the mask to effectively eliminate artifacts, and does not rely on the

¹Conghao Huang, ^{2*}Li Zhang are affiliated with the School of Mathematics, Hefei University of Technology, Hefei, 230601, China. 2023111354@mail.hfut.edu.cn

³Tianchen Deng studies at Shanghai Jiao Tong University. ⁴Kangxu Wang studies at Tsinghua University. ⁵Mingrui Li studies at Dalian University of Technology.

The authors acknowledge partial support from the National Natural Science Foundation of China (Grant 61972131), as well as the National Key Research and Development Program of China (Grant 2018YFB2100301).

*Corresponding author.

accuracy of the mask, significantly enhancing the adaptability of the system.

The following highlights summarize the core innovations introduced in this study:

- MPDG-SLAM is proposed as a 3DGS-powered SLAM solution specifically targeted at handling dynamic scenarios. It is capable of swiftly and effectively mitigating the disturbances caused by dynamic objects while reconstructing the static map, encompassing an MP-based geometric processing module and a versatile rendering solution module.
- We propose an MP-based adaptive penalty and a universal rendering loss, which circumvents the intricate mask-based rendering strategies employed in other methods.
- Our method significantly enhances both tracking and rendering accuracy. Extensive experiments were performed on two real-world datasets, where our method outperformed state-of-the-art approaches, achieving a 95.6% gain in average tracking accuracy and a 72.9% improvement in rendering accuracy.

II. RELATED WORK

A. NeRF-based SLAM in dynamic environments

The superior capability of Neural Radiance Fields (NeRF) [10] in 3D reconstruction has led to increasing interest in NeRF-based SLAM techniques in recent years [11], [12]. iMAP [13], as the first outstanding SLAM solution based on NeRF, demonstrates for the first time the application of multilayer perceptrons (MLP) exclusively representing scenes in online RGB-D SLAM pipelines. However, it performs poorly in real dynamic scenes. With the development of NeRF, more and more researchers are focusing on handling such situations. NID-SLAM [14] generates dynamic masks through optical flow estimation and performs background completion, reducing dynamic noise interference. However, its tracking accuracy is limited, and the rendering process lacks sufficient constraints, affecting overall performance. Although RoDyn-SLAM [15] benefits from combining motion and semantic cues to refine motion estimation, its rendering quality and real-time capabilities are still insufficient. DDN-SLAM [16] combines semantic features, a mixed Gaussian distribution model, and a sparse point cloud sampling strategy to eliminate dynamic interference, but rendering artifacts are limited by the inherent residual errors of implicit representations, which are difficult to fully eliminate. This is also the core problem faced by most NeRF-based SLAM methods.

B. Dynamic SLAM based on 3DGS

Owing to the efficient rendering and detailed 3D reconstruction features of 3D Gaussian Splatting (3DGS) [17]–[19], a variety of high-performing SLAM systems have been developed around this approach. SplaTAM [20] is the first to utilize explicit volumetric representation (i.e., 3D Gaussians) to achieve high-fidelity reconstruction from a single unposed RGB-D camera, mitigating the scene modeling limitations inherent in previous NeRF-based techniques. MonoGS [21],

through innovative camera tracking and geometric verification methods, enables high-fidelity real-time 3D scene reconstruction. However, both methods struggle to operate robustly in dynamic environments, and research addressing this issue remains limited. DG-SLAM [22] improves pose estimation accuracy in dynamic environments and ensures high-quality rendering in static scenes through the use of motion-aware masking, dynamic Gaussian handling strategies, and a combined camera tracking scheme. However, its pose estimation accuracy is still constrained by the precision of semantic prior segmentation. To tackle the GS-SLAM problem in dynamic environments, we leverage motion probability (MP) segmentation of backend Gaussian points and epipolar geometry-based state verification of frontend feature points to effectively filter out anomalous dynamic points. Based on this, we refine the rendering loss and introduce additional constraints to effectively resolve the decoupling problem commonly found in traditional SLAM systems, where tracking and mapping operate independently under dynamic scenarios.

III. METHOD

Our framework is shown in the Fig. 1. In section III-A, we perform an initial static-dynamic classification and labeling of the point cloud based on the computed MP. In section III-B, to improve segmentation accuracy and avoid missing static Gaussian points, we map the labeled points back to the front-end via a Gaussian pyramid and apply epipolar constraint-based verification to ensure more static Gaussian points are densified. In section III-C, leveraging the Gaussian points' motion probabilities, we adjust photometric and depth losses, introduce a regularization term, and fuse a warping constraint to jointly optimize pose estimation and point cloud reconstruction, targeting static scene rendering.

A. Dynamic Gaussian Segmentation Based on MP

In GS-SLAM, each 3D Gaussian is parameterized by a set of 3D attributes, including pose, opacity, scale, and rotation. To better adapt to dynamic environments and achieve more complete rendering results while accurately estimating the camera pose, we add a Motion Probability (MP) attribute to the Gaussian points. Unlike traditional dynamic SLAM methods, we do not directly remove dynamically detected feature points in the front-end. Instead, in the back-end, we label their dynamic/static attributes based on the MP of the Gaussian points, and generate Gaussian masks for dynamic Gaussians. Afterward, we utilize a Gaussian pyramid architecture to relate the 3D Gaussian back to the initial feature points in the front-end, thus eliminating the impact of dynamic features on pose estimation and effectively removing artifacts from the scene map. The Gaussian pyramid here is constructed through multi-scale smoothing and downsampling, creating image layers at different resolutions to capture details at various levels [23].

We use MP to measure the likelihood of an object's motion. Due to the continuous changes in the scene and viewpoint, we dynamically update the MP of the Gaussian

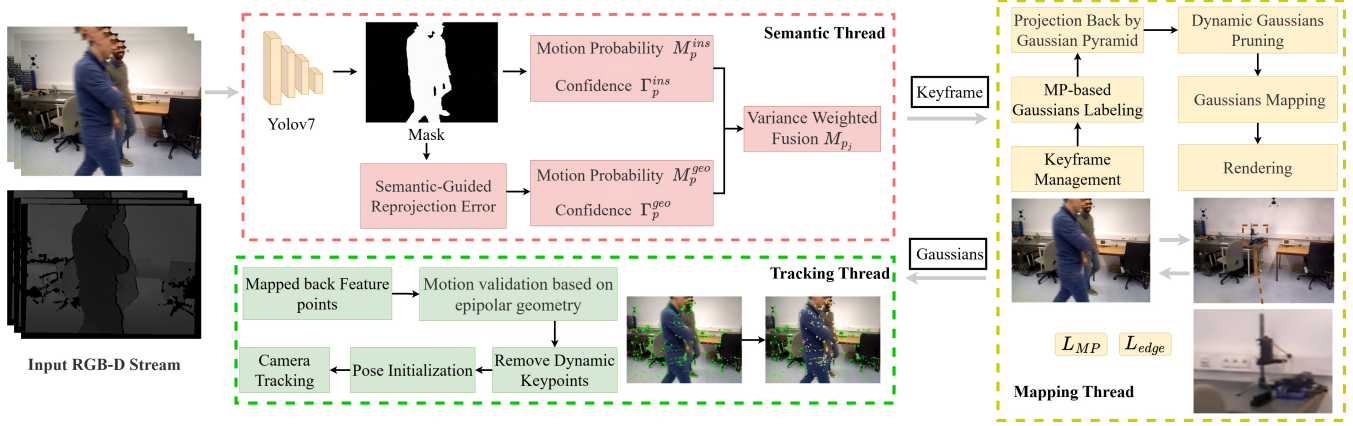


Fig. 1: Pipeline of MPDG-SLAM. We calculate the motion probability of the backend Gaussian points to perform an initial labeling of dynamic Gaussians. These labels are then mapped back to the frontend via a Gaussian pyramid and filtered out using our designed epipolar verification. Based on the motion probability, we further optimize the rendering loss and introduce a new penalty term, ultimately rendering a complete static environment.

point $\mathcal{G}$ using YOLOv7 instance segmentation and semantic-guided geometric methods. Instance segmentation assigns a semantic motion probability MP_{ins} to each feature point (e.g., human 0.8, chair 0.5, small objects on the table 0.3, background 0.2). However, due to the limited prior knowledge, we introduce multi-view reprojection constraints to compute the geometric motion probability MP_{geo} of feature points and optimize the final MP through variance-weighted fusion. The frame with the largest covisibility from historical keyframes and calculate the re-projection error between the current frame's Gaussian point $\mathcal{G}$ and the observed point p_i :

$$e(p_i; R, t) = \|p_i - \pi(R\mathcal{G} + t)\|, \quad (1)$$

where π is the projection function, and R and t represent the camera pose. Within the same instance region, we calculate the variance σ_e and maximum error e_{max} of the projection errors for the sampled points $\{p_i\}_{i=1}^n$, and based on this, we define the geometric motion probability of the feature point p_i within the region:

$$MP_{geo}(p_i) = \begin{cases} \frac{\sum_{i=1}^n e(p_i)}{ne_{max}} & \sigma_e \leq \sigma \\ \frac{e(p_i)}{e_{max}} & \sigma_e > \sigma \end{cases}, \quad (2)$$

where σ is the predefined variance threshold. For points located in the "external" region, since the motion state may be uneven, we directly use:

$$MP_{geo}(p) = \frac{e(p)}{e_{max}}, \quad (3)$$

then the MP of the feature point M_p is obtained by variance-weighted fusion (VWF) of the two results:

$$M_p = MP_{ins} + K(MP_{geo} - MP_{ins}), \quad (4)$$

where K is used to measure the contribution of MP_{ins} and MP_{geo} to the result:

$$K = \frac{\Gamma_p^{geo}}{\Gamma_p^{ins} + \Gamma_p^{geo}}, \quad (5)$$

here, Γ_p^{ins} represents the confidence from instance segmentation, and Γ_p^{geo} is estimated based on the reprojection error variance. The MP of the Gaussian point $\mathcal{G}$ depends on the information of the corresponding feature points observed in each image frame, so M_G can be expressed as a function dependent on the observed feature points M_p :

$$M_G = \frac{1}{m} \sum_{j=1}^m M_{p_j}, \quad (6)$$

where m represents the number of observed feature points.

Finally, considering the set of motion probabilities $\{M(\mathcal{G}_i)\}$ of the Gaussian points observed in the current frame, the median $\tilde{M}$ is defined, and the dynamic threshold τ is set as:

$$\tau = \begin{cases} \tilde{M} & \text{if } \tilde{M} > 0.5 \\ 0.5 & \text{otherwise} \end{cases}, \quad (7)$$

when $M(\mathcal{G}_i) > \tau$, the Gaussian point is labeled as dynamic, otherwise, it is labeled as static.

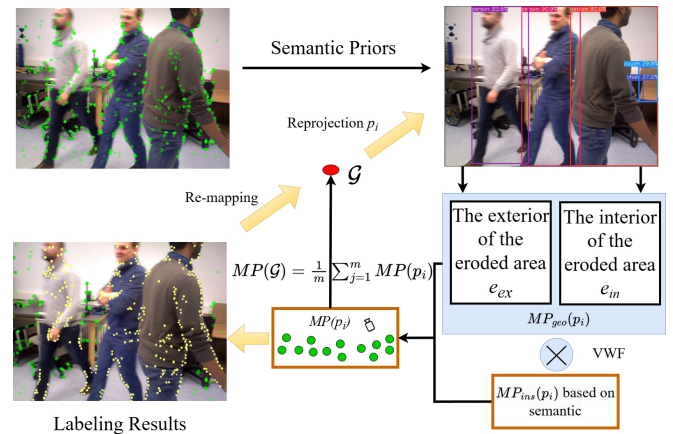


Fig. 2: The labeling process of our method, where yellow represents dynamic outlier points and green represents static feature points.

B. Densification in the Context of Epipolar Geometry

We have marked the dynamic/static attributes of the Gaussian points based on their MP. To improve the robustness of our GS-SLAM system, a simple approach is to directly remove the Gaussian points labeled as dynamic. However, when dynamic objects are highly complex or move drastically, the accuracy of our segmentation method becomes crucial. Many static Gaussians located at the edges of dynamic objects may be misclassified as dynamic, which could lead to holes in the rendered scene, affecting the integrity and continuity of the map.

Following the mapping of Gaussians to front-end feature points through a Gaussian pyramid, a motion verification procedure grounded in epipolar geometry is carried out. Subsequently, we densify and map these to Gaussian points, further identifying missed dynamic Gaussians and restoring static Gaussians that were mistakenly labeled as dynamic. Specifically, we assume that p_1 and p_2 are the matched feature points of the Gaussian point $\mathcal{G}$ in the previous and current frames, and L_1 and L_2 are the epipolar lines in the corresponding frames. The epipolar line L_2 can be computed from the fundamental matrix F as:

$$L_2 = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = Fp_1 = K^{-T}t^{\wedge}RK^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad (8)$$

where X , Y , and Z are the coefficients of the 2D linear equation, K , R , and $t^{\wedge}$ denote the camera's intrinsic matrix, rotation matrix, and the skew-symmetric matrix of the translation vector, respectively. Then the distance d_i between the feature point p_i and the corresponding epipolar line is described as [24]:

$$d_i = \frac{|p_i^T F p_1|}{\sqrt{\|X\|^2 + \|Y\|^2}}. \quad (9)$$

Since dynamic points have not been completely removed in the calculation of the fundamental matrix, this may lead to inaccuracies in the fundamental matrix. To obtain a more accurate fundamental matrix, before performing the second fundamental matrix calculation, we use the distance from the matching points to the epipolar lines to exclude some points that do not meet the constraints. If the sum of the epipolar lines corresponding to these points is below the empirical threshold ϵ , they will be densified and remapped to the Gaussian points through the following Gaussian function:

$$g(p_i) = o \exp\left(-\frac{1}{2}p_i^T \Sigma^{-1} p_i\right), \Sigma = RSS^T R^T, \quad (10)$$

here Σ encodes the covariance information, opacity is determined by o ranging from 0 to 1, with S and R being the scale and rotation matrices. Afterward, they will be labeled as static Gaussians. The labeling process of our method is shown in Fig 2, where the yellow anomalous dynamic points will be filtered out.

C. Rendering and Loss Optimization based on MP

In 3D Gaussian rendering, an explicit point-based scene representation is used for optimization. For Gaussians labeled dynamic, their depth is set to infinity for pruning. However, this may lead to difficult-to-remove artifacts. Therefore, we adjust the original photometric and depth rendering loss with the MP of the Gaussian points as follows:

$$L_{pho} = \sum_i M_{\mathcal{G}}^i \cdot |C_i(\mathcal{G}, T_{CW}) - C_i^{gt}|,$$

$$L_{depth} = \sum_i M_{\mathcal{G}}^i \cdot |D_i(\mathcal{G}, T_{CW}) - D_i^{gt}|, \quad (11)$$

where $C_i(\mathcal{G}, T_{CW})$ renders the Gaussian $\mathcal{G}$ using transformation matrix T_{CW} , $D_i(\mathcal{G}, T_{CW})$ represents depth rasterization, the pose-aligned reference RGB and depth frames are represented by C_i^{gt} and D_i^{gt} , respectively.

Additionally, an MP-based penalty is introduced to further mitigate the rendering effects of truly dynamic Gaussians, effectively reducing occlusion-related artifacts and preventing static Gaussians from being mistakenly eliminated:

$$L_{MP} = \sum_i \|\mathcal{G}_i - \mathcal{G}_i^{est}\|^2 \cdot M_{\mathcal{G}}^i, \quad (12)$$

where $\mathcal{G}_i$ and $\mathcal{G}_i^{est}$ correspond to the actual and predicted positions of the Gaussian point, $M_{\mathcal{G}}^i$ reflects its likelihood of motion. Moreover, dynamic scene mapping typically introduces substantial noise and undermines overall structural consistency. Using only rendering loss for pose refinement increases the risk of tracking errors and susceptibility to disturbances [25]. To this end, we add an edge warping loss that reinforces geometric consistency between adjacent-frame associations. Specifically, let p be a 2D pixel in frame i , we reproject it onto frame j through a distortion operation [15]:

$$f_{warp}(T_{ji}, p_i, D(p_i)) = KT_{ji}(K^{-1}D(p_i)p_i^{homo}), \quad (13)$$

here, D and T_{ji} represent depth information and transformation matrix respectively, p_i^{homo} is the homogeneous coordinate of p_i , then for an edge set $\mathcal{E}_i$ the edge warp loss defines as:

$$L_{edge} = \sum_{p_i \in \mathcal{E}_i} \rho(D_j(f_{warp}(T_{ji}, p_i, D(p_i)))) \cdot M_p^j, \quad (14)$$

where Huber kernel function ρ is employed to attenuate the impact of residual outliers in the optimization process. We apply varying weights λ_i to control how motion probability affects tracking and mapping. Then the final rendering loss can be described by minimizing the objective function shown below:

$$L_G = \lambda_1 \cdot L_{photo} + \lambda_2 \cdot L_{depth} + \lambda_3 \cdot L_{MP} + \lambda_4 \cdot L_{edge}. \quad (15)$$

TABLE I: Evaluation of Absolute Trajectory Error (ATE) on the TUM RGB-D Dataset.

Top-performing results are annotated as **first**, **second**, "X" represents cases where tracking failed. All metrics are measured in [m].

Sequences	ORB-SLAM3		Photo-SLAM		SplaTAM		DDN-SLAM		DG-SLAM		MPDG-SLAM	
	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD
fr3/waking/xyz	0.3728	0.1967	0.3052	0.1481	0.4324	0.138	0.0147	0.0097	0.0165	0.0087	0.0129	0.0081
fr3/waking/xyz_val	0.6823	0.5126	0.6145	0.3987	0.8847	0.8097	0.0113	0.0070	0.0214	0.0109	0.0130	0.0055
fr3/waking/static	0.1768	0.0967	0.1348	0.0117	0.3469	0.4327	0.0117	0.0059	0.0063	0.0100	0.0068	0.0033
fr3/waking/static_val	0.2785	0.0712	0.2632	0.0729	0.7369	0.4289	0.0137	0.0080	0.0209	0.0312	0.0110	0.0086
fr3/waking/rpy	0.6130	0.2697	0.5642	0.2790	X	X	0.0420	0.0267	0.0436	0.0210	0.0319	0.0291
fr3/waking/rpy_val	0.3429	0.1983	0.2980	0.1780	X	X	0.0279	0.0160	0.0390	0.0216	0.0267	0.0170
fr3/waking/half	0.2320	0.1239	0.2671	0.1314	0.8901	0.6233	0.0213	0.0113	0.0299	0.0338	0.0230	0.0100
fr3/waking/half_val	0.4670	0.1907	0.3213	0.2017	1.3429	1.0568	0.0278	0.0257	0.0286	0.0380	0.0225	0.0140
Avg.	0.3957	0.2075	0.2991	0.1777	0.7723	0.5811	0.0204	0.0141	0.0271	0.0218	0.0174	0.0113

TABLE II: Evaluation of Absolute Trajectory Error (ATE) on the BONN RGB-D Dataset.

Top-performing results are annotated as **first**, **second**, "X" represents cases where tracking failed. All metrics are measured in [m].

Sequences	ORB-SLAM3		Photo-SLAM		SplaTAM		DDN-SLAM		DG-SLAM		MPDG-SLAM	
	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD
crowd	0.4301	0.0211	0.5102	0.0112	1.829	1.238	0.0193	0.0112	0.0210	0.0182	0.0179	0.0091
crowd2	0.9163	0.5578	0.9329	0.5786	3.4561	1.871	0.0228	0.0110	0.0320	0.0244	0.0230	0.0126
person_tracking	0.6122	0.3221	0.3291	0.4521	0.5291	0.1020	0.0429	0.0169	0.0459	0.0341	0.0421	0.0180
person_tracking2	0.6567	0.5567	0.3246	0.2679	0.6608	0.4267	0.0376	0.0129	0.0691	0.0389	0.0399	0.0220
synchronous	0.7671	1.2380	0.4563	0.5643	X	X	0.0301	0.0290	0.0489	0.0285	0.0281	0.0212
synchronous2	1.5453	1.0068	0.6890	0.7765	X	X	0.0187	0.0130	0.0209	0.0128	0.0179	0.0140
balloon	0.0630	0.0467	0.0339	0.0175	2.768	1.328	0.0179	0.0089	0.0371	0.0117	0.0189	0.0102
balloon2	0.4569	0.7821	0.6420	0.6679	1.2391	0.9860	0.0421	0.0278	0.0410	0.0341	0.0402	0.0194
Avg.	0.6466	0.6039	0.4929	0.4258	1.6788	0.9934	0.0291	0.0158	0.0383	0.0236	0.0286	0.0157

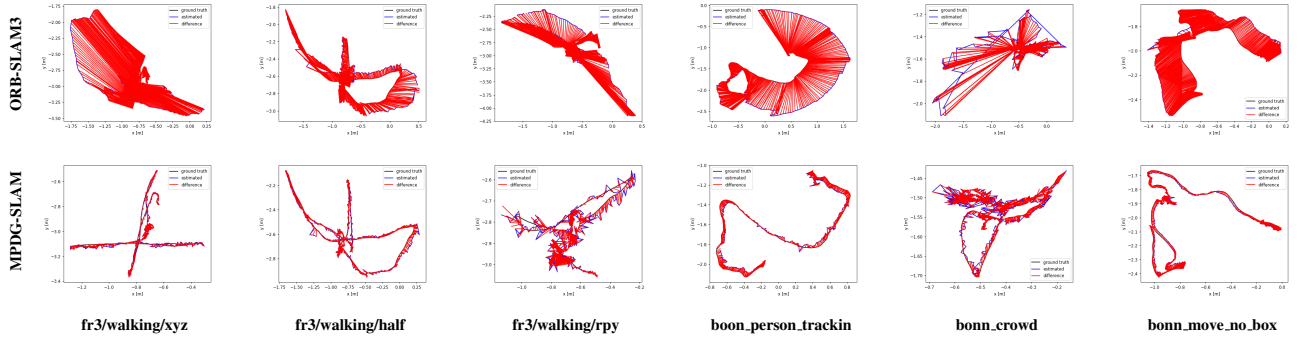


Fig. 3: Comparison of trajectories estimated by ORBSLAM3 and MPDG-SLAM against ground truth.

IV. EXPERIMENTS

a) Datasets: We test our method on the TUM RGB-D [26] and BONN datasets [27] to demonstrate its effectiveness in practical scenarios, the latter of which has higher dynamics and a large number of undefined dynamic objects. Both datasets are captured using handheld devices.

b) Metrics: We assess tracking accuracy using two indicators derived from the absolute trajectory error (ATE): RMSE (root mean squared deviation), STD (standard deviation) [26]. We measure scene reconstruction fidelity using three key metrics—accuracy and completeness, both in centimeters (cm), along with the completeness ratio (< 5 cm %), while ground truth data provided by the BONN dataset.

Additionally, to quantitatively analyze the view synthesis results, we report several representative photometric metrics: (i) peak signal-to-noise ratio (PSNR), which evaluates per-pixel correctness; (ii) structural similarity index (SSIM), reflecting perceptual geometric integrity; (iii) learned perceptual image patch similarity (LPIPS), which captures high-level visual similarity, where ground truth values are derived from the reconstructed images. We use #param (M) and GPU (G) to measure computational load and memory usage. Furthermore, to test whether the system meets real-time operational requirements, frames per second(FPS) and per-frame latency(ms) are used as performance metrics.

c) Implementation: The MPDG-SLAM experiments are conducted on a system equipped with Ubuntu 18.04, Intel

TABLE III: Quantitative Evaluation of Mapping quality on Dynamic Scenes from the BONN dataset, which are highlighted as **first**, **second**.

		ball	ball2	person_tracking	person_tracking2	move_no_box2	Avg.
Co-SLAM	Acc.[cm]↓	16.70	31.19	35.41	54.22	17.12	30.90
	Comp.[cm]↓	27.29	40.32	201.40	118.47	20.47	77.27
	Comp. Ratio[≤ 5cm%]↑	3.11	3.30	1.98	2.85	24.91	16.31
ESLAM	Acc.[cm]↓	17.22	26.78	59.20	89.34	12.27	41.04
	Comp.[cm]↓	9.09	13.60	145.81	186.69	10.11	73.10
	Comp. Ratio[≤ 5cm%]↑	25.73	21.86	20.49	17.29	41.38	34.87
NID-SLAM	Acc.[cm]↓	10.58	14.53	26.58	25.87	12.69	17.97
	Comp.[cm]↓	10.71	30.09	124.92	107.32	10.19	60.79
	Comp. Ratio[≤ 5cm%]↑	47.71	43.97	11.61	12.59	40.01	19.29
RoDyn-SLAM	Acc.[cm]↓	10.78	13.33	10.45	13.81	11.56	11.98
	Comp.[cm]↓	7.21	7.90	27.69	18.87	6.79	13.68
	Comp. Ratio[≤ 5cm%]↑	47.60	40.89	34.07	32.61	45.40	40.21
Ours(MPDG-SLAM)	Acc.[cm]↓	9.89	11.97	11.79	12.29	10.03	10.82
	Comp.[cm]↓	6.76	8.81	22.91	15.04	7.48	11.87
	Comp. Ratio[≤ 5cm%]↑	49.94	43.83	39.80	36.52	43.21	42.66

TABLE IV: The view synthesis results as the mapping quality for dynamic sequences in the BONN dataset, the results are highlighted as **first**, **second**.

		ball	ball2	person_tracking	person_tracking2	move_no_box2	Avg.
ESLAM	PSNR.[dB]↑	15.51	15.38	14.85	18.81	16.86	16.88
	SSIM.[%]↑	0.643	0.663	0.819	0.646	0.796	0.766
	LPIPS.[%]↓	0.530	0.558	0.344	0.417	0.579	0.487
RoDyn-SLAM	PSNR.[dB]↑	15.75	16.68	17.96	19.40	15.33	17.42
	SSIM.[%]↑	0.592	0.609	0.828	0.659	0.674	0.680
	LPIPS.[%]↓	0.397	0.417	0.325	0.340	0.501	0.371
DG-SLAM	PSNR.[dB]↑	17.84	18.60	20.57	19.79	19.89	18.13
	SSIM.[%]↑	0.721	0.857	0.891	0.752	0.884	0.767
	LPIPS.[%]↓	0.367	0.435	0.201	0.310	0.457	0.378
MPDG-SLAM	PSNR.[dB]↑	21.41	22.21	25.27	23.18	21.59	22.73
	SSIM.[%]↑	0.887	0.732	0.951	0.857	0.851	0.855
	LPIPS.[%]↓	0.327	0.176	0.351	0.165	0.241	0.252

i7 and RTX 3090 hardware components. In the experiments, the empirical threshold ϵ is set to 1 according to [7], the variance threshold σ is initialized to 6, real-time updates of the camera pose are achieved using the Levenberg-Marquardt algorithm [28], and the loss weights are set to $\lambda_1 = 0.9$, $\lambda_2 = 0.1$, $\lambda_3 = 500$, $\lambda_4 = 300$.

d) Baseline Methods: To assess the system’s effectiveness in scene reconstruction and camera localization, we adopt the following benchmarks: Photo-SLAM [23], OVD-SLAM [29], NID-SLAM [14] and DDN-SLAM [16]. At the same time, we will conduct comparative experiments and ablation studies in real dynamic scenarios with advanced methods: ORB-SLAM3 [30], SplaTAM [20], DG-SLAM [22], RoDyn-SLAM [15], Co-SLAM [31], ESLAM [32], DN-SLAM [33], NICE-SLAM [34] and baseline methods.

A. Comparison with State-of-the-arts on Tracking

We benchmark our method against advanced baselines on the TUM RGB-D dataset to evaluate its tracking perfor-

mance in dynamic conditions. As shown in TABLE I, we combine the backend Gaussian point-based MP dynamic-static segmentation with the frontend epipolar constraint-based state validation, which effectively eliminate the impact of dynamic elements on pose estimation, achieving high tracking accuracy. Moreover, as shown in TABLE II, in the more dynamically challenging BONN dataset, our method still outperforms the majority of advanced methods. Additionally, to better visualize the global effect of our method during the tracking process, the camera trajectories and their ground-truth counterparts are visualized for both datasets to highlight differences, as shown in Fig. 3.

B. Evaluation of Gaussian Mapping

In dynamic environments, SLAM based on 3DGS generates Gaussians for dynamic objects, leading to incomplete reconstruction of static scenes. We use a qualitative evaluation method based on the scene’s view synthesis performance to assess its impact on Gaussian mapping, as shown in

Fig. 4. And TABLE IV presents the quantitative evaluation of mapping quality based on the synthesized views across dynamic scenes in the BONN dataset. Compared to existing advanced Gaussian-based rendering SLAM methods, such as DG-SLAM, our approach effectively eliminates dynamic artifacts as much as possible, clearly rendering scene elements occluded by dynamic objects. Additionally, we conduct a quantitative comparison with other advanced methods on the BONN dataset using common reconstruction metrics, with results presented in TABLE III. From the experimental results, it is evident that our proposed MPDG-SLAM achieves the best performance among Gaussian splatting SLAM methods in dynamic scenes of the datasets.



Fig. 4: Visualization depicting reconstructions on the benchmark dataset.



Fig. 5: Rendering Ablation Results

C. Ablation Study

We investigate how each component of our system affects both tracking performance and Gaussian representation of the environment, we conduct ablation experiments on a public dataset. w/o MP indicates that motion probabilities derived from backend Gaussian points are not employed for dynamic Gaussian labeling; w/o geo means that after mapping the

labeled Gaussian points back to the frontend, we do not perform further optimization with epipolar geometry; Under the w/o backend condition, our system bypasses backend Gaussian point association, opting for direct elimination of dynamic frontend features via classical geometric and semantic analysis.

For the ablation on the tracking part, based on the TABLE V, the proposed approach achieves enhanced pose estimation accuracy due to its tight integration of Gaussian mapping and dynamic element filtering in the camera tracking module. For the mapping part, we perform ablation experiments on these components using PSNR, SSIM, as shown in the TABLE VI. Our method, which fully considers dynamic Gaussians in the backend, reconstructs a higher-quality map compared to other advanced methods. Additionally, we separately study the impact of w/o backend on rendering to highlight the innovation of our method over traditional methods. The visual comparisons in Fig. 5 highlight the improved rendering fidelity and detail preservation achieved by our approach.

TABLE V: Tracking ablation study of the proposed method on the datasets, the best result is highlighted as **first**.

Dataset	Metric	w/o MP	w/o geo	w/o backend	MPDG-SLAM
BONN	ATE ↓	0.5217	0.0451	0.0315	0.0212
	STD ↓	0.3413	0.0276	0.0197	0.0135
TUM	ATE ↓	0.3521	0.0322	0.0230	0.0202
	STD ↓	0.2819	0.0198	0.0226	0.0171

TABLE VI: Mapping quality ablation results in the BONN Dataset, the best results are highlighted as **first**.

		crowd	person_tracking	move_no_box	Avg.
w/o MP	PSNR,[dB]↑	13.63	18.36	17.32	16.43
	SSIM,[%]↑	0.612	0.913	0.730	0.751
w/o geo	PSNR,[dB]↑	14.78	17.91	19.21	17.30
	SSIM,[%]↑	0.519	0.816	0.657	0.664
w/o backend	PSNR,[dB]↑	17.01	18.87	19.62	18.50
	SSIM,[%]↑	0.718	0.829	0.837	0.794
MPDG-SLAM	PSNR,[dB]↑	22.45	24.89	20.67	22.67
	SSIM,[%]↑	0.708	0.923	0.885	0.839

D. Comprehensive Analysis

As shown in the TABLE VII, we evaluate the system's average runtime, mapping time, GPU memory usage. According to the experimental findings, the method not only accelerates execution and lowers memory demands but also maintains reliable performance accuracy.

V. CONCLUSIONS

We propose MPDG-SLAM, a dynamic-resistant method that tightly integrates camera tracking with Gaussian mapping. It jointly utilizes dynamic-static segmentation based on Gaussian points (MP) and state validation based on epipolar geometry, and optimizes the Gaussian rendering module based on this. Compared to other methods, it achieves the best tracking results and more accurate rendering, while also

TABLE VII: Execution comprehensive comparisons on the BONN dataset reveal the leading method, with the most efficient and second-most efficient entries specially highlighted **first**, **second** for contrast.

Method	Mapping(ms)↓	FPS↑	#param.↓	GPU(G)↓
NICE-SLAM	291.8	1.2	11.9	10.9
DN-SLAM	61.9	4.2	30.1	5.8
DDN-SLAM	50.7	20.1	19.6	6.2
MPDG-SLAM	48.9	32.1	10.2	6.4

enabling deployment on mobile devices with lower computational power consumption. It improves pose estimation accuracy and achieves complete, clear reconstruction of the static environment. In future work, we will focus on applying this method to low-power.

REFERENCES

- [1] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19595–19604, 2024.
- [2] T. Deng, Y. Chen, L. Zhang, J. Yang, S. Yuan, J. Liu, D. Wang, H. Wang, and W. Chen, "Compact 3d gaussian splatting for dense visual slam," *arXiv preprint arXiv:2403.11247*, 2024.
- [3] S. Zhou, H. Chang, S. Jiang, Z. Fan, Z. Zhu, D. Xu, P. Chari, S. You, Z. Wang, and A. Kadambi, "Feature 3dgs: Supercharging 3d gaussian splatting to enable distilled feature fields," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21676–21685, 2024.
- [4] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai, "Scaffold-gs: Structured 3d gaussians for view-adaptive rendering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20654–20664, 2024.
- [5] M. Kong, J. Lee, S. Lee, and E. Kim, "Dgs-slam: Gaussian splatting slam in dynamic environment," *arXiv preprint arXiv:2411.10722*, 2024.
- [6] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, "Flowfusion: Dynamic dense rgb-d slam based on optical flow," in *2020 IEEE international conference on robotics and automation (ICRA)*, pp. 7322–7328, IEEE, 2020.
- [7] S. Cheng, C. Sun, S. Zhang, and D. Zhang, "Sg-slam: A real-time rgb-d visual slam toward dynamic scenes with semantic and geometric information," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–12, 2022.
- [8] S. Wu, X. Zhang, S. Zhang, Z. Song, R. Wang, and J. Yuan, "Mpoc-slam: An rgb-d slam system with motion probability and object category in high dynamic environments," *IEEE/ASME Transactions on Mechatronics*, 2024.
- [9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 7464–7475, 2023.
- [10] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [11] M. Li, J. He, Y. Wang, and H. Wang, "End-to-end rgb-d slam with multi-mlps dense neural implicit representations," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7138–7145, 2023.
- [12] Z. Wang, S. Wu, W. Xie, M. Chen, and V. A. Prisacariu, "Nerf-: Neural radiance fields without known camera parameters," 2021.
- [13] Z. Xu, J. Niu, Q. Li, T. Ren, and C. Chen, "Nid-slam: Neural implicit representation-based rgb-d slam in dynamic environments," in *2024 IEEE International Conference on Multimedia and Expo (ICME)*, pp. 1–6, IEEE, 2024.
- [14] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "imap: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 6229–6238, 2021.
- [15] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, "Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields," *IEEE Robotics and Automation Letters*, 2024.
- [16] M. Li, Y. Zhou, G. Jiang, T. Deng, Y. Wang, and H. Wang, "Ddn-slam: Real-time dense dynamic neural implicit slam," *arXiv preprint arXiv:2401.01545*, 2024.
- [17] M. Li, S. Liu, and H. Zhou, "Sgs-slam: Semantic gaussian splatting for neural dense slam," in *European Conference on Computer Vision*, Springer, Cham, 2025: 163–179., 2024.
- [18] H. Zhou, Z. Guo, S. Liu, L. Zhang, Q. Wang, Y. Ren, and M. Li, "Mod-slam: Monocular dense mapping for unbounded 3d scene reconstruction," *IEEE Robotics and Automation Letters*, 2024.
- [19] M. Li, S. Liu, T. Deng, and H. Wang, "Densesplat: Densifying gaussian splatting slam with neural radiance prior," *arXiv preprint arXiv:2502.09111*, 2025.
- [20] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "Splatam: Splat track & map 3d gaussians for dense rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21357–21366, 2024.
- [21] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18039–18048, 2024.
- [22] Y. Xu, H. Jiang, Z. Xiao, J. Feng, and L. Zhang, "Dg-slam: Robust dynamic gaussian splatting slam with hybrid pose optimization," *arXiv preprint arXiv:2411.08373*, 2024.
- [23] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21584–21593, 2024.
- [24] Y. Zhuang, P. Jia, Z. Liu, L. Li, C. Wu, X. Lu, W. Cui, and Z. Liu, "Amos-slam: an anti-dynamics two-stage rgb-d slam approach," *IEEE Transactions on Instrumentation and Measurement*, vol. 73, pp. 1–10, 2023.
- [25] H. Li, X. Gu, W. Yuan, L. Yang, Z. Dong, and P. Tan, "Dense rgb slam with neural implicit maps," *arXiv preprint arXiv:2301.08930*, 2023.
- [26] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 573–580, IEEE, 2012.
- [27] E. Palazzolo, J. Behley, P. Lottes, P. Giguere, and C. Stachniss, "Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7855–7862, IEEE, 2019.
- [28] A. Ranganathan, "The levenberg-marquardt algorithm," *Tutorial on LM algorithm*, vol. 11, no. 1, pp. 101–110, 2004.
- [29] J. He, M. Li, Y. Wang, and H. Wang, "Ovd-slam: An online visual slam for dynamic environments," *IEEE Sensors Journal*, vol. 23, no. 12, pp. 13210–13219, 2023.
- [30] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," *IEEE transactions on robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [31] H. Wang, J. Wang, and L. Agapito, "Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13293–13302, 2023.
- [32] M. M. Johari, C. Carta, and F. Fleuret, "Eslam: Efficient dense slam system based on hybrid representation of signed distance fields," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17408–17419, 2023.
- [33] C. Ruan, Q. Zang, K. Zhang, and K. Huang, "Dn-slam: A visual slam with orb features and nerf mapping in dynamic environments," *IEEE Sensors Journal*, vol. 24, no. 4, pp. 5279–5287, 2023.
- [34] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12786–12796, 2022.